

```

from pyspark.sql import SparkSession
import pyspark.sql.functions as F
from pyspark.sql.types import DoubleType
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import MultilayerPerceptronClassifier
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
from pyspark.ml.evaluation import BinaryClassificationEvaluator

print("1. Εκκίνηση αυτόνομης SparkSession για Neural Networks (Διορθωμένο)...")
spark = SparkSession.builder \
    .appName("MLP_SMOTE_Corrected_GridSearch") \
    .master("local[*]") \
    .config("spark.driver.memory", "8g") \
    .getOrCreate()

print("2. Φόρτωση Δεδομένων (Απευθείας από το SMOTE Gold dataset)...")
train_gold = spark.read.parquet("../data/train_gold_with_cluster.parquet")
test_gold = spark.read.parquet("../data/test_gold_with_cluster.parquet")

train_gold = train_gold.withColumn("stroke",
train_gold["stroke"].cast(DoubleType()))
test_gold = test_gold.withColumn("stroke",
test_gold["stroke"].cast(DoubleType()))
train_gold = train_gold.withColumn("cluster",
F.col("cluster").cast(DoubleType()))
test_gold = test_gold.withColumn("cluster", F.col("cluster").cast(DoubleType()))

print("3. Feature Augmentation (Ενσωμάτωση K-Means Cluster)...")
assembler = VectorAssembler(inputCols=["features", "cluster"],
outputCol="augmented_features")

train_augmented =
assembler.transform(train_gold).drop("features").withColumnRenamed("augmented_features",
"features")
test_augmented =
assembler.transform(test_gold).drop("features").withColumnRenamed("augmented_features",
"features")

train_augmented.cache()
test_augmented.cache()

print("4. Υπολογισμός Διαστάσεων Εισόδου (Input Size)...")
input_size = len(train_augmented.select("features").first()[0])
print(f" -> Βρέθηκαν {input_size} χαρακτηριστικά εισόδου (Features + Cluster).")

print("5. Ορισμός Neural Network και ΔΙΟΡΘΩΜΕΝΟΥ Πλέγματος Παραμέτρων...")
mlp_base = MultilayerPerceptronClassifier(featuresCol="features",
labelCol="stroke", seed=22390225)

```

```

# ΟΙ ΑΛΛΑΓΕΣ: Πολύ πιο "στενά" δίκτυα (λιγότεροι νευρώνες) και λιγότερες
επαναλήψεις
# Αυτό αποτρέπει την απομνημόνευση των συνθετικών δεδομένων του SMOTE!
paramGrid = (ParamGridBuilder()
    .addGrid(mlp_base.layers, [
        [input_size, 8, 4, 2],      # Στενό δίκτυο 2 κρυφών επιπέδων
        [input_size, 6, 2]          # Ακραία απλό δίκτυο 1 κρυφού
    ])
    .addGrid(mlp_base.maxIter, [40, 80]) # Σταματάμε την εκπαίδευση
    νωρίς (Early Stopping)
    .build())

evaluator = BinaryClassificationEvaluator(
    labelCol="stroke",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderPR"
)

cv = CrossValidator(estimator=mlp_base,
    estimatorParamMaps=paramGrid,
    evaluator=evaluator,
    numFolds=3,
    seed=22390225)

print("6. Εκτέλεση Cross-Validation στο SMOTE Dataset...")
cv_model = cv.fit(train_augmented)
best_mlp = cv_model.bestModel

print("\n" + "="*60)
print("[ΕΥΡΕΣΗ ΠΑΡΑΜΕΤΡΩΝ NEURAL NETWORK ΟΛΟΚΛΗΡΩΘΗΚΕ]")
print(f" -> Βέλτιστη Αρχιτεκτονική (Layers): {best_mlp._java_obj.getLayers()}")
print(f" -> Βέλτιστος αριθμός Επαναλήψεων (maxIter): {best_mlp._java_obj.getMaxIter()}")
print("="*60 + "\n")

print("7. Παραγωγή προβλέψεων στο άγνωστο Test Set...")
mlp_preds = best_mlp.transform(test_augmented)

print("8. Αποθήκευση των τελικών προβλέψεων...")
output_path = "../data/mlp_predictions.parquet"
mlp_preds.select("stroke", "prediction",
    "probability").write.mode("overwrite").parquet(output_path)

print(f"=====")
print(f"[ΕΠΙΤΥΧΙΑ] Το Διορθωμένο Neural Network εκπαιδεύτηκε και αποθηκεύτηκε!")
print(f"=====")

```

```
spark.stop()
```

1. Εκκίνηση αυτόνομης SparkSession για Neural Networks (Διορθωμένο)...
2. Φόρτωση Δεδομένων (Απευθείας από το SMOTE Gold dataset)...
3. Feature Augmentation (Ενσωμάτωση K-Means Cluster)...
4. Υπολογισμός Διαστάσεων Εισόδου (Input Size)...
-> Βρέθηκαν 22 χαρακτηριστικά εισόδου (Features + Cluster).
5. Ορισμός Neural Network και ΔΙΟΡΘΩΜΕΝΟΥ Πλέγματος Παραμέτρων...
6. Εκτέλεση Cross-Validation στο SMOTE Dataset...

```
=====
[EΥΡΕΣΗ ΠΑΡΑΜΕΤΡΩΝ NEURAL NETWORK ΟΛΟΚΛΗΡΩΘΗΚΕ]
-> Βέλτιστη Αρχιτεκτονική (Layers): [I@5d22d06b
-> Βέλτιστος αριθμός Επαναλήψεων (maxIter): 80
=====
```

7. Παραγωγή προβλέψεων στο άγνωστο Test Set...
8. Αποθήκευση των τελικών προβλέψεων...

```
=====
[ΕΠΙΤΥΧΙΑ] Το Διορθωμένο Neural Network εκπαιδεύτηκε και αποθηκεύτηκε!
=====
```